Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
Пермский институт (филиал) РЭУ им. Г.В. Плеханова

Контрольная работа

МДК _____ / ОП ____ «Основы проектирования баз данных»

Специальность ИКТ «Информационно-коммуникационные технологии»

Тема: Разработка модуля учёта склада на WPF с использованием MVVM и коллекций.

Листов: 14

Руководитель
Д.Б.Берестов /И.О. Фамилия/
«___24___» сентября_ 2026 г.

Исполнитель
П.И. Правденкова /И.О. Фамилия/
«___24___» сентября_ 2026 г.

2026

Содержание

Теоретическая часть 3
Практическая часть 4
Анализ технического задания 4
Создание проекта 5
Трудности в процессе разработки 11
Юнит-тестирование 11
Вывод 14

Теоретическая часть

Visual Studio представляет собой мощную интегрированную среду разработки, созданную компанией Microsoft для создания программного обеспечения на различных языках программирования, включая C#, VB.NET, C++, Python и другие. В рамках данной среды можно разрабатывать проекты разных типов, например, консольные приложения, приложения с графическим интерфейсом на базе Windows Presentation Foundation (WPF), веб-приложения на ASP.NET Core, а также проекты модульных тестов. WPF является фреймворком для создания клиентских приложений под управлением Windows, который использует язык разметки XAML для описания интерфейса и позволяет применять паттерн Model-View-ViewModel (MVVM). MVVM разделяет приложение на три слоя: модель (данные и бизнес-логика), представление (интерфейс) и модель представления (ViewModel), которая связывает данные и команды с интерфейсом с помощью привязок данных (data binding). Такой подход упрощает тестирование и сопровождение кода. Для хранения данных в учебных проектах часто используют обычные коллекции C#, такие как List<T>, что позволяет отказаться от развёртывания полноценной базы данных. Модульное тестирование в Visual Studio реализуется с помощью встроенного фреймворка MSTest, а также поддерживаются NUnit и xUnit. Тесты позволяют проверить каждый метод сервиса в изоляции, выявить ошибки на ранних этапах и обеспечить надёжность приложения. Таким образом, использование Visual Studio, WPF, MVVM и модульного тестирования даёт разработчику полный набор инструментов для создания качественного программного продукта.

Практическая часть

Анализ технического задания

Из ТЗ я выделила ключевые требования: три взаимосвязанных модуля – организации, склады, товары. Организации можно добавлять, редактировать, удалять с подтверждением через ввод названия. Склады привязываются к выбранной организации. Товары привязываются к выбранному складу и отображаются в таблице с фото, категорией, производителем, поставщиком, ценой, остатком, скидкой. Дополнительно – создание накладных (приходных и расходных), которые меняют остаток товара. Весь интерфейс должен показывать текущий контекст: какая организация и какой склад выбраны. Также нужны клиентские валидации и обработка ошибок.

Я думала сделать окна так же раздельно, но в последствии полной переделки из-за потери файлов, при отключении света, подумала, что логичнее сделать не три отдельных окна, как нарисовано в ТЗ, а одно главное окно, где слева организации, справа склады и товары. Так пользователю будет нагляднее, и контекст не потеряется.

Структура решения: три проекта. Проект Warehouse – само WPF-приложение с окнами и ViewModel. Проект Warehouse.Core – библиотека классов, где лежат модели (Организация, Склад, Товар, Накладная), сервисы для CRUD-операций и интерфейсы к ним. Проект Warehouse.Tests – модульные тесты для сервисов и для проверки валидации в окнах.

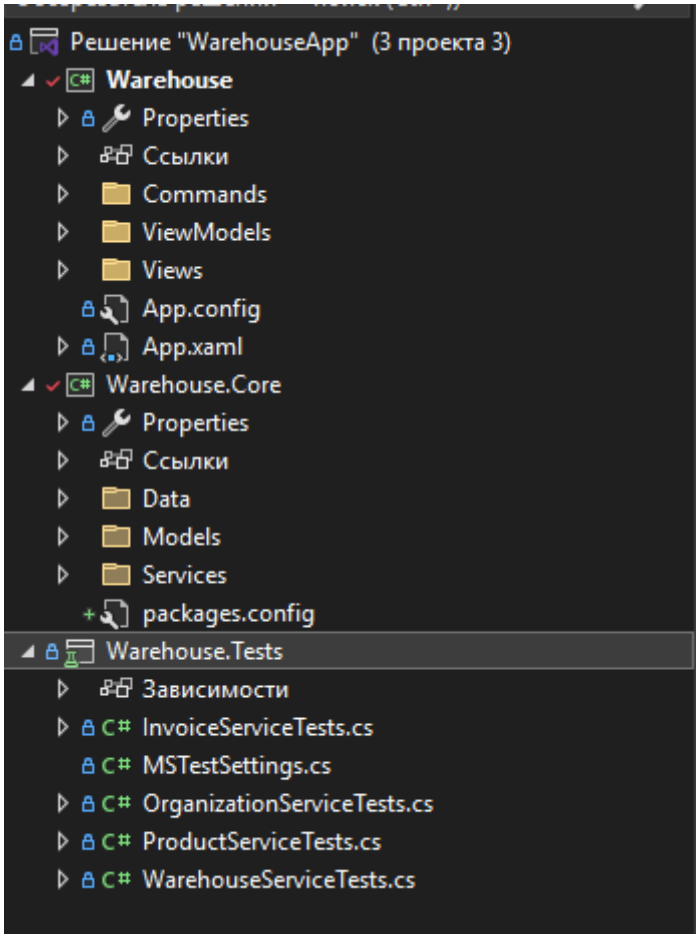
Создание проекта

Первым делом были созданы классы в папке Models проекта Warehouse.Core. Организация имеет Id и Name. Склад – Id, Name, Address, OrganizationId. Товар — Id, Name, Category, Manufacturer, Supplier, Price, StockQuantity, DiscountPercent, ImagePath, WarehouseId. Накладная – Id, WarehouseId, Type, Date и список позиций, где ProductId, Quantity, UnitPrice. Сразу было продумано, что остаток товара будет изменяться только через накладные, а напрямую редактировать остаток в товаре можно, но это не имеет смысла, лучше делать накладные.

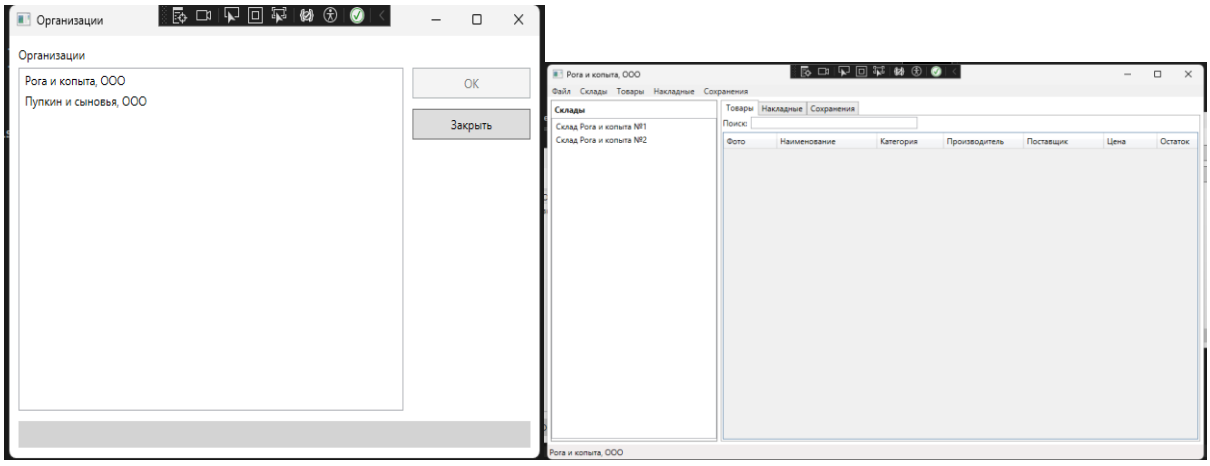
Позже, когда писались сервисы, стало понятно, что Id нужно назначать автоматически: находится максимальный Id в списке и прибавляется единица. В проекте Warehouse.Core был создан статический класс DataStore. В нём находятся публичные статические списки для организаций, складов, товаров и накладных. В статическом конструкторе добавлены две организации: «Рога и копыта, ООО» и «Пупкин и сыновья, ООО».

Для каждой созданы склады: для первой один склад, для второй два склада. Также добавлены товары для демонстрации. Потом в этом же классе созданы экземпляры сервисов, которым переданы списки. Так получилось, что при запуске приложения данные уже есть.

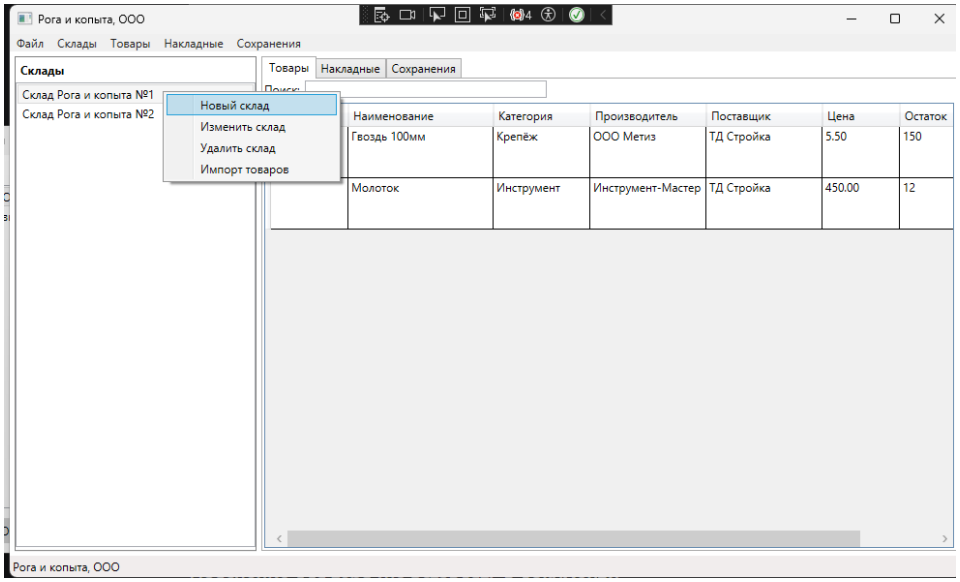
Были написаны интерфейсы IOrganizationService, IWarehouseService, IProductService, IInvoiceService. В каждом есть методы GetAll, Add, Update, Delete, а также специфические, например GetByOrganizationId для складов. Классы-реализации принимают в конструкторе соответствующий список. В методах работа идёт просто со списком. В InvoiceService метод CreateInvoice не только добавляет накладную в список, но и изменяет StockQuantity у товара. Если накладная расходная и товара не хватает, выбрасывается исключение с понятным сообщением.



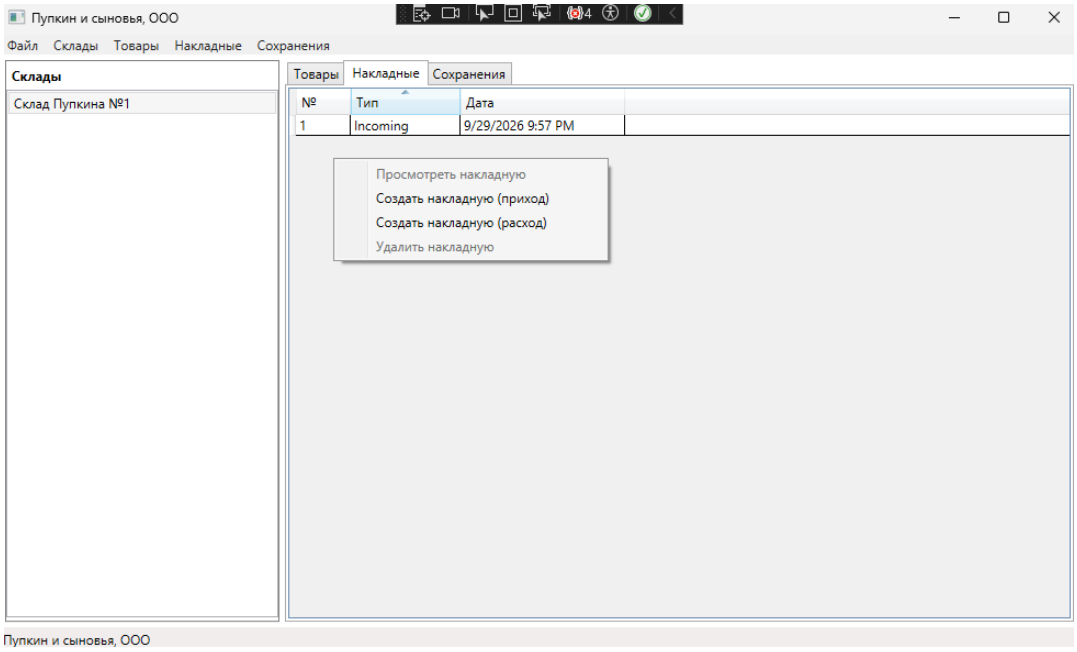
Интерфейс приложения построен на двух окнах. Первое окно – главное, MainWindow.xaml. В нём только список организаций, кнопки «OK» и «Закрыть», а также контекстное меню, которое вызывается правой кнопкой мыши по списку и содержит пункты «Новая организация», «Изменить организацию» и «Удалить организацию». При выборе организации в списке она становится контекстом для дальнейшей работы. По нажатию «OK» главное окно не закрывается, а открывается второе окно – WarehouseManageWindow.xaml, в котором и происходит управление складом.

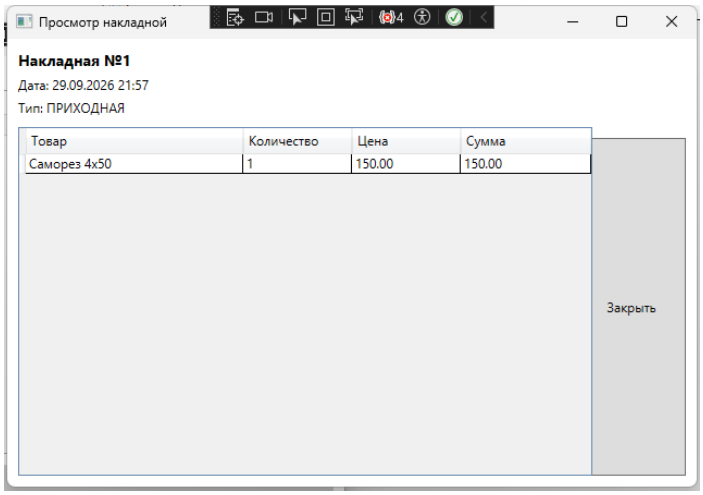


Второе окно, WarehouseManageWindow.xaml, открывается уже с названием выбранной организации в заголовке. Сверху расположено меню, в котором есть разделы «Файл», «Склады», «Товары», «Накладные» и «Сохранения». Эти пункты всегда активны и работают независимо от того, что выбрано в данный момент. Слева находится панель со списком складов, привязанных к выбранной организации. По правой кнопке мыши по этому списку открывается контекстное меню с пунктами «Новый склад», «Изменить склад», «Удалить склад» и «Импорт товаров».

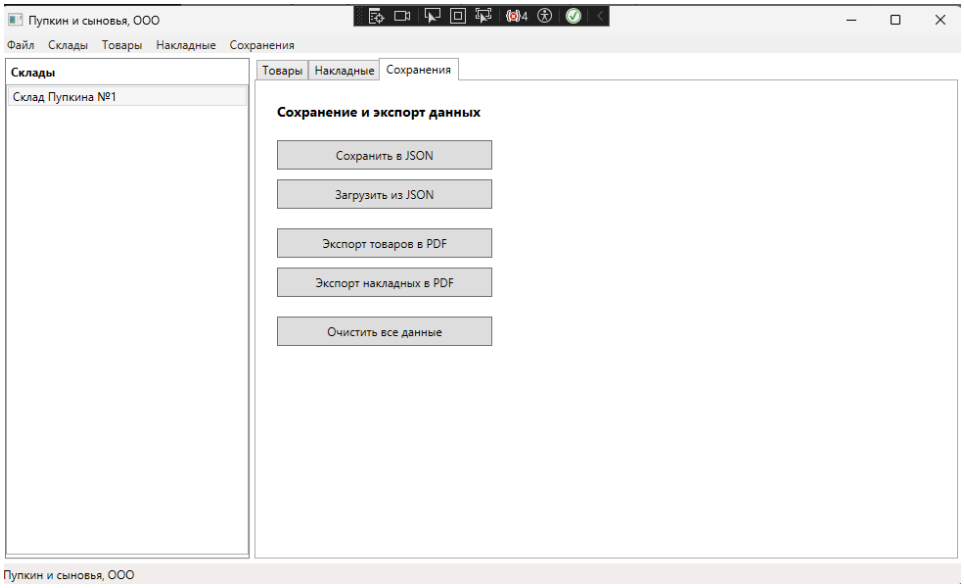


Правая часть второго окна выполнена в виде вкладок. Вкладка «Товары» содержит поле поиска и таблицу товаров с колонками «Фото», «Наименование», «Категория», «Производитель», «Поставщик», «Цена», «Остаток» и «Скидка %». Вкладка «Накладные» содержит таблицу созданных накладных с колонками «Номер», «Тип» и «Дата», а по правой кнопке мыши и по двойному клику можно открыть окно просмотра накладной, где видно её состав: наименование товара, количество, цена за единицу и сумма.

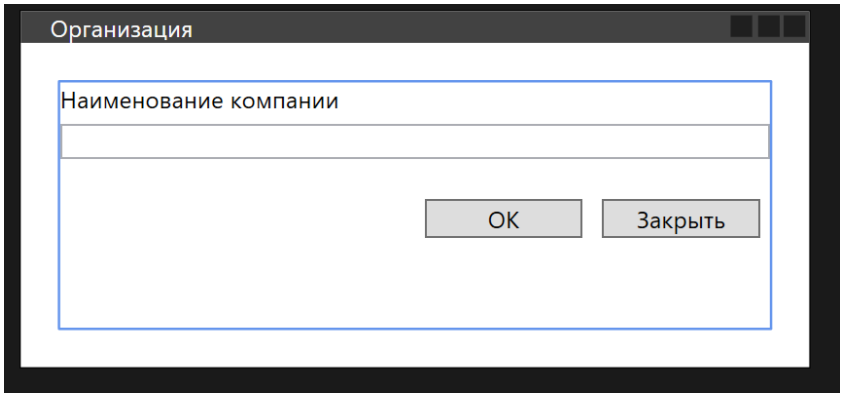


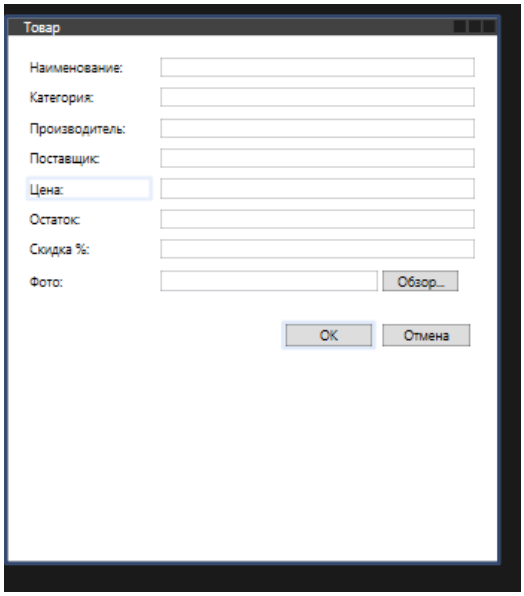


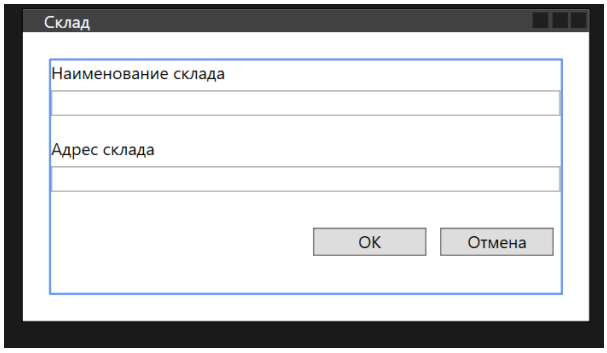
Вкладка «Сохранения» содержит кнопки для сохранения данных в JSON, загрузки из JSON, экспорта товаров в PDF, экспорта накладных в PDF и очистки всех данных.

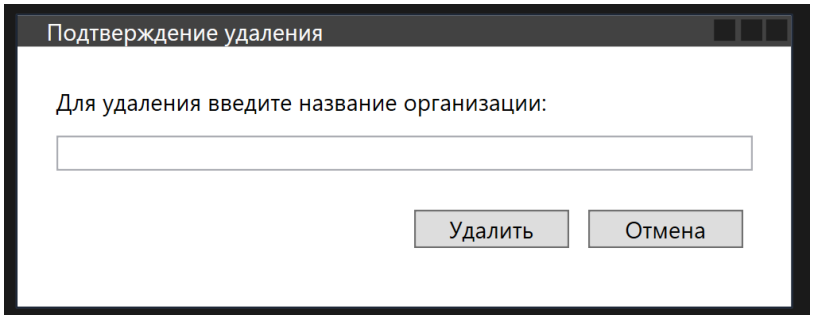


Окна для добавления, редактирования я сделала отдельными: AddEditOrganizationWindow, AddEditWarehouseWindow, AddEditProductWindow, CreateInvoiceWindow. Также окно подтверждения удаления организации – DeleteConfirmationWindow.

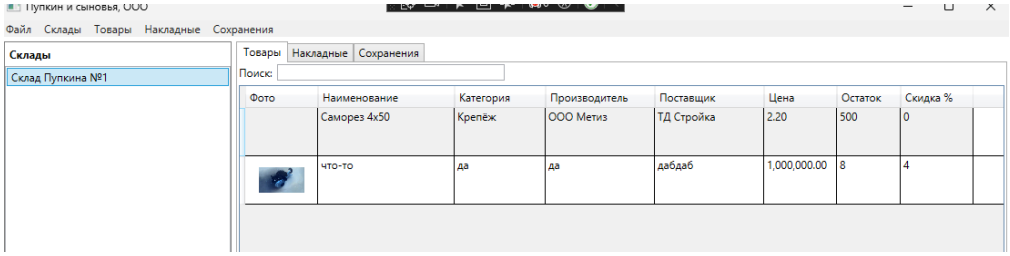




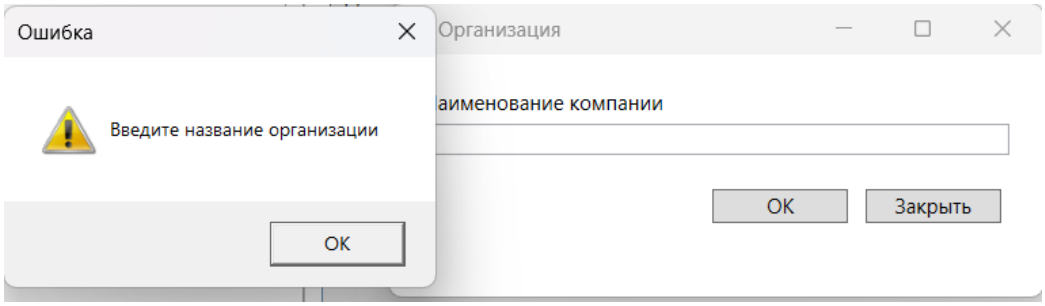




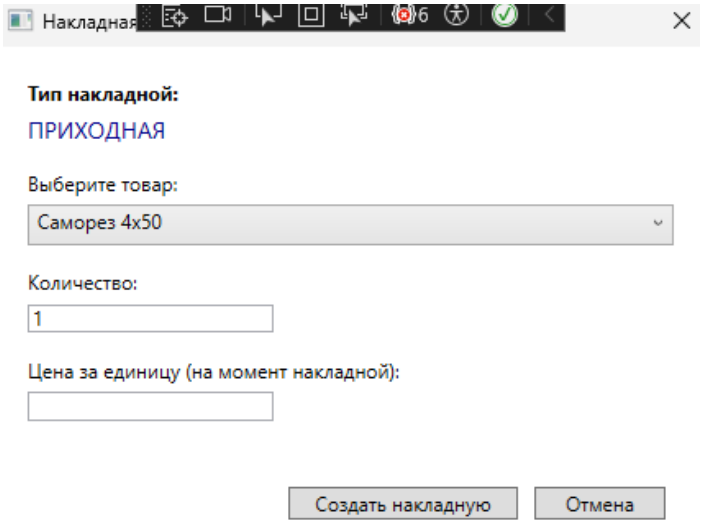
В окне AddEditProductWindow я добавила поле для фото и кнопку «Обзор...». При нажатии открывается OpenFileDialog, выбирается картинка, путь сохраняется в ImagePath. В DataGrid в главном окне я использовала DataGridTemplateColumn с Image, чтобы картинка отображалась. Если файла по пути нет – ничего страшного, просто пусто.



Я уделила внимание валидации. При нажатии OK в каждом окне проверяю: не пустые ли обязательные поля, корректны ли числа. Если ошибка – показываю MessageBox и не закрываю окно. Так пользователь сразу понимает, что ввёл неверно.



В MainViewModel сделаны два метода CreateIncomingInvoice и CreateOutgoingInvoice. Они вызывают окно CreateInvoiceWindow, которому передаётся выбранный склад, тип накладной, список товаров на этом складе и сервис накладных. В окне пользователь выбирает товар, вводит количество и цену за единицу. При создании накладной сервис проверяет, хватает ли товара для расхода, и изменяет остаток. После успеха DataGrid товаров обновляется. Если накладная на новый товар, достаточно создать одну единицу товара и через накладную уже добавить остальное.



Трудности в процессе разработки

Когда я создавала класс Warehouse в модели, у меня возник конфликт: проект тоже назывался Warehouse, и компилятор путался. В какой-то момент студия перестала видеть класс Warehouse в Warehouse.Core. Я просто решила поменять название и добавила в него цифру 1, не стала заморачиваться ища другие обходы.

Еще изначально я создала проекты под .NET Framework 4.8. Но в последствии некоторые упрощенные версии написания кода, которые я знала не работали на framework, я поискала решения проблемы в интернете и просто выгрузила проект, и изменила файл, добавив версию 9.0 Core, это за секунду решило все проблемы и код стало писать еще проще. Так как на этой версии удобней лично мне.

Из неприятного в какой-то момент мой ПК неожиданно отключился не сохранив должно решение, из-за чего не сохранились некоторые файлы, а половину файлов и вовсе решение не видело, вручную добавляла имеющиеся файлы, а пустые пересоздавала заново.

Юнит-тестирование

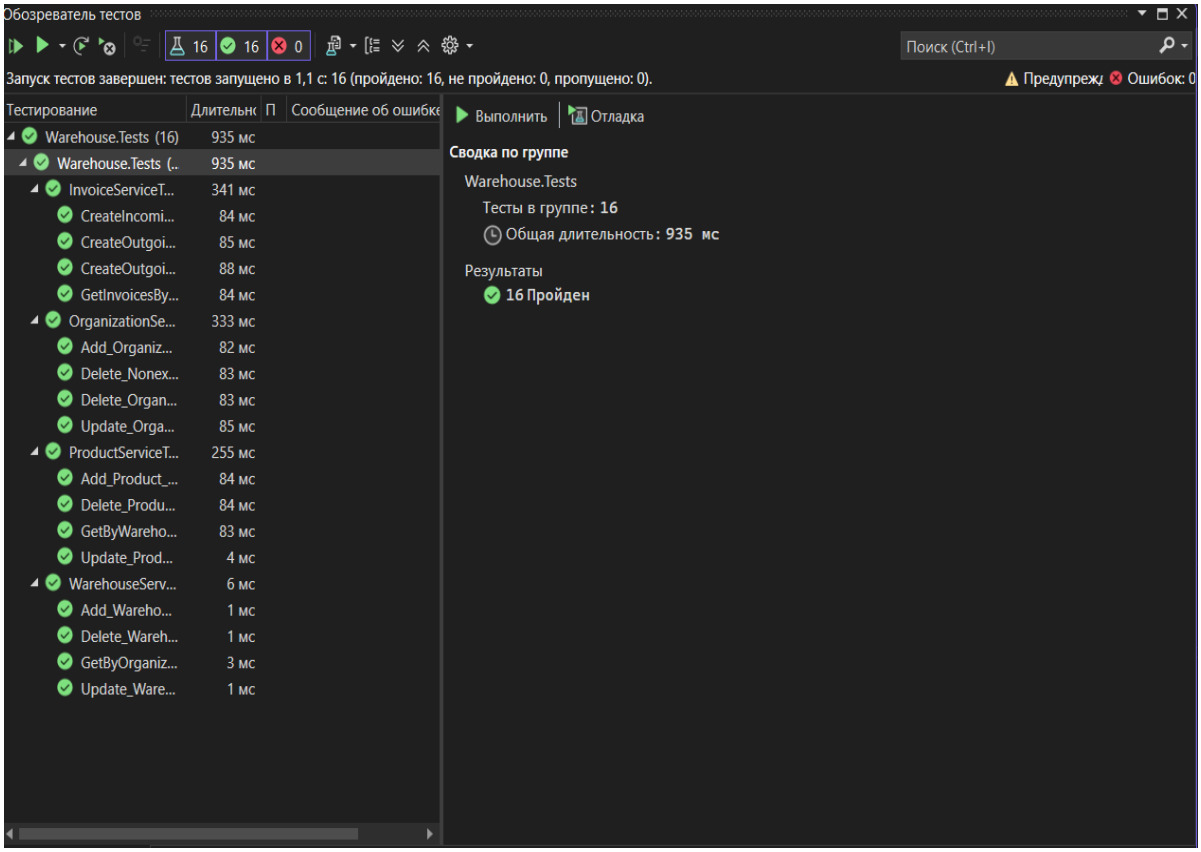
Я разработала тесты для каждого сервиса в отдельном проекте Warehouse.Tests с использованием фреймворка MSTest, который встроен в Visual Studio.

Для **OrganizationService** я создала четыре тестовых метода. Первый тест проверяет добавление новой организации – он добавляет организацию с именем "Тест" и проверяет, что количество элементов в списке увеличилось на единицу, а также что новой записи автоматически присвоился идентификатор 1. Второй тест проверяет обновление – берётся существующая организация, её имя изменяется, и после вызова метода Update тест убеждается, что в списке сохранилось новое имя. Третий тест удаляет организацию по идентификатору и проверяет, что список стал пустым. Четвёртый тест проверяет, что при попытке удалить несуществующий идентификатор ничего не происходит и количество записей не меняется.

Для **WarehouseService** я написала тесты, которые проверяют фильтрацию складов по идентификатору организации – метод GetByOrganizationId должен возвращать только те склады, которые принадлежат указанной организации. В одном из тестов я создаю три склада, два из которых относятся к организации 10, и проверяю, что метод возвращает ровно два элемента. Далее проверяется добавление склада с автоматическим назначением Id, обновление его имени и адреса, а также удаление.

ProductService тестируется аналогично: получение товаров по идентификатору склада, добавление, обновление всех полей (наименование, категория, производитель, поставщик, цена, остаток, скидка, путь к фото), удаление. Особое внимание я уделила проверке, что при обновлении все свойства действительно изменяются.

InvoiceService – самый важный с точки зрения бизнес-логики. Первый тест создаёт приходную накладную на товар, у которого изначальный остаток 10 единиц, и проверяет, что после проведения накладной остаток стал 15 (увеличился на количество). Второй тест делает расходную накладную на 3 единицы – остаток уменьшается с 10 до 7. Третий тест проверяет, что при попытке списать больше товара, чем есть на складе (например, 5 единиц при остатке 2), метод CreateInvoice выбрасывает исключение с сообщением о недостатке товара. Это важно для защиты от ошибок учёта. Четвёртый тест проверяет, что метод GetInvoicesByWarehouse возвращает только накладные, относящиеся к конкретному складу.


Обозреватель тестов
16
16
0
Поиск (Ctrl+I)
Запуск тестов завершен: тестов запущено в 1,1 с: 16 (пройдено: 16, не пройдено: 0, пропущено: 0).
Предупрежд
Ошибок: 0
Тестирование
Длительно
П
Сообщение об ошибке
Warehouse.Tests (16) 935 мс
Warehouse.Tests (.. 935 мс
InvoiceServiceT... 341 мс
CreateIncomi... 84 мс
CreateOutgoi... 85 мс
CreateOutgoi... 88 мс
GetInvoicesBy... 84 мс
OrganizationSe... 333 мс
Add_Organiz... 82 мс
Delete_Nonex... 83 мс
Delete_Organ... 83 мс
Update_Orga... 85 мс
ProductServiceT... 255 мс
Add_Product_... 84 мс
Delete_Produ... 84 мс
GetByWareho... 83 мс
Update_Prod... 4 мс
WarehouseServ... 6 мс
Add_Wareho... 1 мс
Delete_Wareh... 1 мс
GetByOrganiz... 3 мс
Update_Ware... 1 мс
Выполнить
Отладка
Сводка по группе
Warehouse.Tests
Тесты в группе: 16
Общая длительность: 935 мс
Результаты
16 Пройден

Вывод

В ходе работы я создала полностью функционирующее приложение «Управление складом», которое соответствует техническому заданию. Я научилась работать с WPF и MVVM, использовать коллекции вместо базы данных, писать юнит-тесты и тестировать UI через рефлексию. Пришлось преодолеть несколько трудностей – конфликт имён, смена версии .NET, восстановление проекта после сбоя. Теперь я увереннее чувствую себя в C# и понимаю, как важно планировать архитектуру заранее, как избегать сбоя, на каких сайтах искать материалы, хотелось сделать понятнее, поэтому я стараюсь совершенствовать работу.